INDEX

ANITS

S.No.	Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
1.	1/7	Introduction & basic HTML	CO1	10	
2.	8/7	Design HTML using audio	CO1	10	
3.	15/7	Design HTML using video	CO1	10	
4.	22/7	Program using JS Carrays	CO1	10	
5	29/7	Develop dynamic webpage	CO1	10	
6.	5/8	Implement Modules in nodejs	CO2	10	
7.	12/8	Mini application using Express & nodejs	CO2	10	
8	19/8	Code Tantra	CO1	10	
9.	2/9	Code Tantra	CO1	10	
10	9/9	Code Trantra	CO1	10	
11	16/9	Create a function & class	CO2	10	
12.	23/9	Implement https methods	CO2	10	
13	30/9	Develop CURD operation	CO3	10	

Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222, Website: www.anits.edu.in email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	~~P.Cha~~ 2026-2027	**Semester:**	1st
Name of the Student:	P. Chaitrika Kalyani	**Roll Number:**	A24126552164
Name of the Lab:	Full stack Lab	**Timings:**	9:40-12:10

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva-Voce (2Marks)			
1/7	1	4	4	5	2	15		
8/7	2	4	3	5	2	14		
15/7	3	4	4	5	2	15		
22/7	4	4	4	5	2	15		
29/7	5	4	4	5	2	15		
5/8	6	4	3	5	2	14		
12/8	7	4	3	5	2	14		
19/8	8	4	4	5	2	15		
23/9	9	4	4	5	2	15		
30/9	10	4	4	5	2	15		

Name of the Faculty: Prabavathy

Faculty Signature:

Introduction to HTML & CSS :-

HTML (Hypertext markup language) is a standard markup language used to creati & structure content on web pages. It difines the basic structure of webpage using elements & tags such as headings, paragraphs, links, images, lists, tables, forms & other multimedia content. HTML documents are interpted by web Browsers to display the content in structured & meaningful manner. HTML provides structured & semantic organization of webpage.

CSS (Cascading style sheets) is a style sheet language used to control the presentation & visual appearance of HTML Document. It is used to define properties such as colors, fonts, background, spacing, boarders, alignment, dimensions & layouts. CSS also supports responsive disign, allowing webpages to adapt to different screen sized & devices CSS can applied using inline, internal or external. HTML & CSS work togather to develop webpages, whre HTML provides the structure & content, while CSS provides styling.

The combination of both, forms the foundation of modern web development & enables the curtain of organised,

atteractive & user friendly web interfaces.

Difference b/w MERN stack & FULLstack.

MERN stack is a java script based technology Stack used for duveloping modern web applications. MERN stack stands for mango DB, Expres js, Ruact js, Node js, is used for data base management, Express js; handles the back end framework & API's, Ruact-js is used to develop frontend user Interface & node-js provides server side run time environment.

→ Full stack Development refers to development of both frontend & backend of web application along with database integration. A Full stack developer works with technologies.

HTML Elements :-

1. HTML tags - Define the structure & elements of webpage.
2. Heading - used to display titles & heading using <h1> to <h6>
3. paragraphs - used to display text using <p> tag.
4. hyperlinks - used to navigate blw webpages using <a> tags.
5. Images - used to display images using <image> tag
6. Lists - used to creati ordered & unordered list using <ol>, <ul>,
7. Tables - Used to organise data in rows & columns.
8. Forms - Used to collect user i/p using elements such as text boxes, buttons & chuck boxes.
9. Div and span - Used to group & organise web page content.
10. Semantic Elements - Elements such as <header>, <nav>, <section>, <article>, <footer> provides meaningful webpage structure.

CSS Elements :-

1. Selectors - select HTML elements to apply styles.
2. properties - Define the visual characteristics of elements
3. values - specify the setting of css property.
4. Colours - Define text, background, boarders, colors.

8/7/26

HTML Elements :-

1. HTML tags - Define the structure & elements of webpage.
2. Heading - used to display titles & heading using <h1> to <h6>
3. paragraphs - used to display text using <p> tag.
4. hyperlinks - used to navigate blw webpages using <a> tags.
5. Images - used to display images using <image> tag
6. Lists - used to create ordered & unordered list using <ol>, <ul>,
7. Tables - Used to organise data in rows & columns.
8. Forms - Used to collect user i/p using elements such as text boxes, buttons & chick boxes.
9. Div and span - Used to group & organise web page content.
10. Semantic Elements - Elements such as <header>, <nav>, <section>, <article>, <footer> provides meaningful webpage structure.

CSS Elements :-

1. Selectors - select HTML elements to apply styles.
2. properties - Define the visual characteristics of elements
3. values - specify the setting of css property.
4. Colours - Define text, background, boarders, colors.

5. Fonts - Control font family, size, style & weight.
6. Background - set background colors & images.
7. Borders - Define border style, width & radius.
8. Margins - Controls space outside an element.
9. padding - Controls space inside an element
10. Layout - Controls arrangement of elements using flexbox, grid & positioning.

HTML vs HTML5 :-

HTML	HTML5
• Older version of HTML	• Latest major version of HTML
• Limited Multimedia support	• Supports audio & video.
• Uses external plugins for Multimedia	• No external plugs needed
• Has fewer semantic tags.	• Provides semantic tags
• No <canvas> element	• Support <canvas>
• Limited webpages	• Supports local storage

CSS VS CSS 3 :-

CSS	CSS 3
. Earlier version of CSS . Basic Styling feature	. Advanced version of CSS provides advanced styling features
. Limited animations	. supports animations & transitions.
. No flexible layout system	. Supports flexbox & grid

15/7/26

Design a static webpage using HTML & CSS

-> Pomodoro Timer - using HTML, CSS

```
<!Doctype HTML>
<html>
<head>
    <title pomodoro timer </title>
    <style>
body {
        font-family: Arial;
        background: #f5f5f5;
        text-align : center;
        padding : 50px;
    }
timer {
        width : 300 px;
        margin : auto;
        padding : 30px;
        back-ground : white;
        border-radius : 15px;
        box-shadow : 0 0 10px grey;
    }

h1 {
        color : #e74c3c;
    }

time {
        font-size : 50px;
        margin : 20px;
}
```

```
button {
        padding : 10px 20px;
        margin : 5px;
        border : none;
        border-radius : 5px;
        background : #e74c3c;
        color : white;
    }

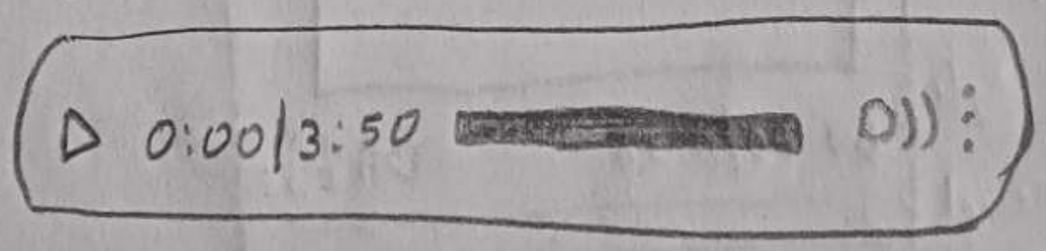
</style>
</head>
<body>
    <div class = "timer">
    <h1> pomodoro timer </h1>
    <div class ="time" > 25:00 </div>
    < button > Start </button>
    < button > Reset </ button >
    </div>
</body>
</html>.
```

1. Design an HTML code using audio tags:-

Code:-

```
<!Doctype html>
<html>
<head>
   <title> Audio Example </title>
</head>
<body>
    <h2> my favorite song </h2>
    <audio controls>
        <source src="song.mp3" type="audio/mpeg">
        Your browser dosnot support audio tag.
    </audio>
</body>
</html>
```

O/P:- Displays an audio player with play/pause, & volume controls.

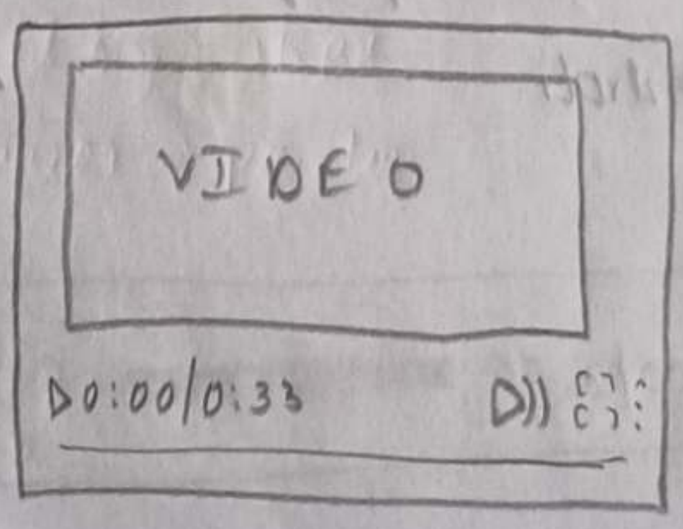


2 Design a HTML code using video tag

Code:-

```
<!Doctype html>
<html>
<head>
   <title> Video example </title>
</head>
<body>
   <h2> My video </h2>
   <video width = "400" controls >
      <source src = "video.mp4" type = "video/mp4">
   </video>
</body>
</html>
```

O/P:- Displays a video player with play, pause, volume, & full & screen control.



3 Write a program using Java Script (Arrays, Func

code:-

```
let numbers = [12, 45, 8, 23, 56, 69, 34, 67, 90, 11];
Function display (arr) {
        Console.log ("Array Element:");
        for (let i=0; i<arr.length; i++)
        {
            Console.log (arr[i]);
        }
}
function findsum (arr) {
        let sum=0;
        for (let i=0; i<arr.length; i++){
            sum += arr[i];
        }
        return sum;
}
```

O/P:-

```
12
45
8
23
56
69
34
67
90
11
Sum: 365
```

Experiment-4

4. Develop dynamic webpage using java script

Program:-

```
<!DOCTYPE Html>
<html>
<head>
<title> Dynamic Webpage </title>
<style>
    body {
        text-align : center;
        font-family : Arial;
        margin-top : 100 px
    }
button {
    padding : 10 px 20 px
    font size : 16 px
    cursor : pointer
}
</style>
</head>
<body>
    <h1 id = "heading"> welcome to my webpage </h1>
    <p id = "Message"> click the button to change this
        webpage </p>

<button onclick = "changePage()"> Click me </button>
```

```
<Script>
    function changePage() {
        document.getElementById("heading").
            innerHTML = "Welcome to javascript!";

        document.getElementById("message").
            innerHTML = "The webpage has changed dynamically!";
        document.body.style.backgroundColor = "lightblue";
    }
</Script>
</body>
</html>
```

O/P:-

Welcome to My Webpage

Click the button to change this webpage

[Click me]

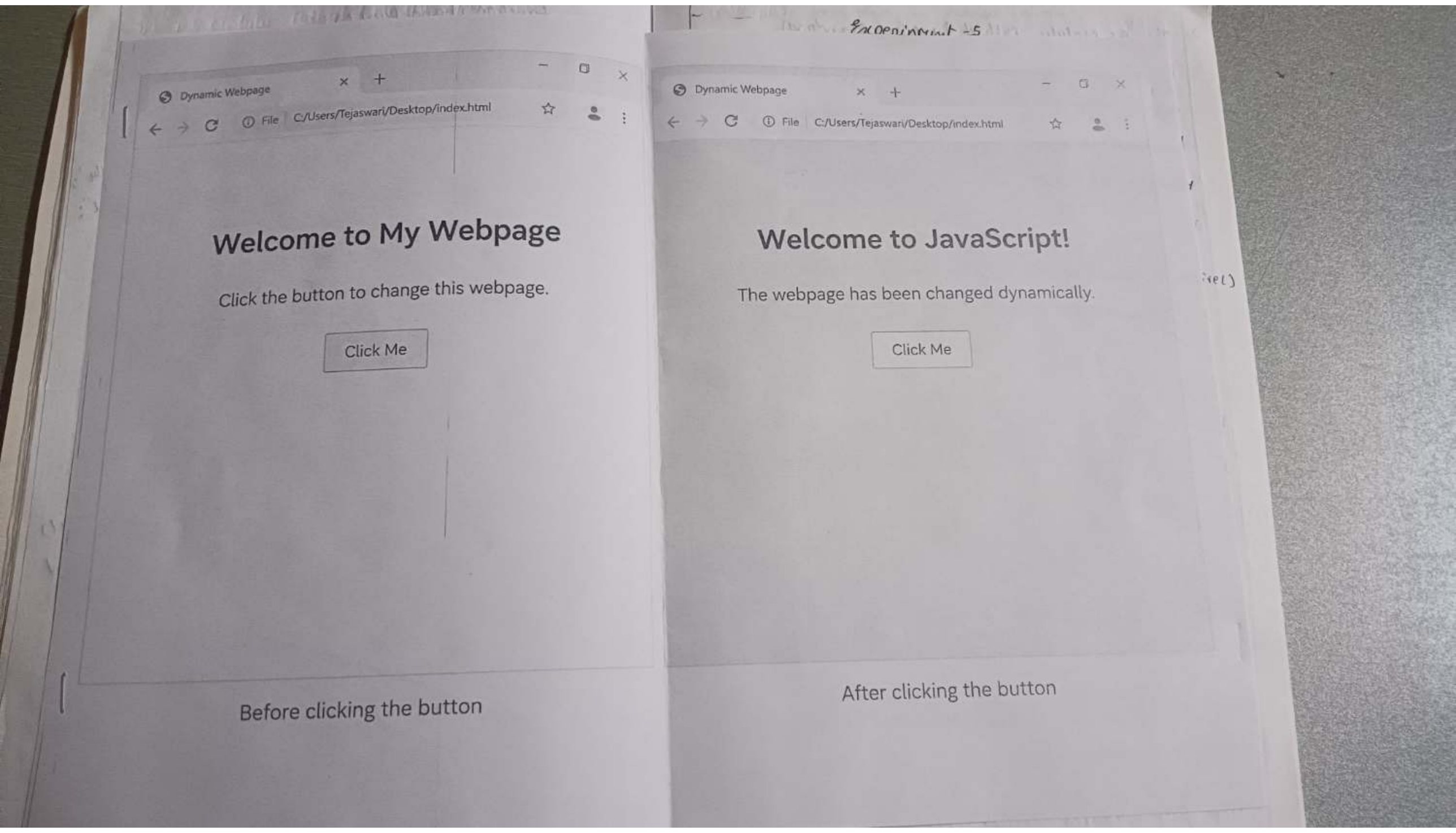
- When user clicks, "Click me", the JS function changePage() is called, then heading dynamically changes & background also changes.

After clicking button:

Welcome to Javascript!

The webpage has been changed dynamically.

[Click me]



Before clicking the button

After clicking the button

Experiment - 5

5. Implement Modules in NodeJs

. In Node.js, you can implement modules using two distinct system : Common JS (CJS), which is traditional default, & ES modules (ESM), the modern JS standard

1. The Common JS method (default):

Common JS uses module.exports to share code & require() to import it. This system loads modules synchronously.

a. Create the module (math.js):-

```
Const add = (a, b) => a+b;
Const subtract = (a, b) => a-b;
Module.exports = {
    add,
    subtract
};
```

b. Use the module (app.js):-

```
Const math = require('./math');
Console.log (math.add (10,5));
Console.log (math.subtract (10,5));
```

O/P:- 15
5

2. The ES Modules method (Modern Standard)

ES modules use export to share code & import to use it
TO make Node.js recognize ES modules, You must either
use the .mjs file extension or add "type": "module" to
Your package.json file

a. Create the module (calculator.js):

```
Export const Multiply = (a, b) => a * b
Export const divide = (a, b) => a/b
Const calculatorName = "superCalc";
Export default CalculatorName;
```

b. Use the module (main.js)

```
import calculatorName, {multiply, divide} from './Kalculator.j
Consol.log (calculatorName);
Consol.log (multiply (4,2));
Consol.log (divide(4,2));
```

O/P:
```
superCalc
8
2
```

3. Using Built-in Core modules:-

Node.js also comes equipped with dozens of built-in core
Modules (like fs for file system or path for file routing)
that don't need installation.

We will load them by module name.

```
// common JS style
const fs = require('fs');
// Es modules style
import fs from 'fs';
```

Here are some of most commonly at ones.

1. fs = file system operations
2. https = HTTP server & client
3. path = file path utilities
4. os = operating system utilities
5. events = event handling
6. stream = Stream handling
7. util = utility function
8. crypto = cryptographic function.
9. url = URL parsing
10. querystring = URL query string handling.

Eg:- Simple HTTPS server

```
https.createServer((req,res) => {
        res.writeHead(200, {'Content-Type':
                                'text/html'});
        res.end('Hello World!');
    3) listen(8080);
```

O/P:- Hello World!

Experinment-6.

6. Develop mini application using expres & Node JS

Simple mini appli. using Node JS & Express JS

Student Management System

1. Add a student
2. View all students
3. delete a student
4. Uses Express JS routes
5. Simple HTML interface

Project structure:-

```
Student.oppl
  ├─ server.js
  └─ public
       └─ index.html
```

Server.js:-

```
Const expres = require ("expres");
Const app = expres();
Const PORT = 3000;
let students = [];
app.use (expres.json());
app.use (expres.static ("public"));
app.get ("/students", (req, res) => {
        res.json (students).
});
//Add
```

```
app post ("Istudents", (req, res) => {
    const student = {
        id. students length + 1;
        nama: req. body. nama;
        Course: req. body. Course
    };
    Students. push (Student);
    res. json ({ message: "student added successfully",
                Student: student });
});

//Delete
app. delete ("/Students/:id", (req, res) => {
    Const id = parseInt (req. params. id);
    Students = students. filter (student =>
                student. id != = id);
    res. json({
        Message: "Student deleted succesfully"});
});

//Start server
app listen (PORT, () => {
    Consol. log (`server running of http://localhost:
                ${PORT}`);
});
```

public index html:-

```
<!DOCTYPE html>
<html>
<head>
    <title> Student Management </title>
</head>
<body>
    <h1> Student Management System </h1>
    <input type = "text" id = "name"
      placeholder = " Enter student name" >
    <input type = "text" id = "course"
      placeholder = " Enter course" >
    <button onclick = "add student()" > Add Student
                                        </button>
<h2> student List</h2>
<ul id = "Student list "> </UL>
<script>
    function addstudent() {
      let name = document.getElementById ("name").value;
      let course=document.getElementById ("course").value;
    fetch ("/students", {
                method: "POST",
                headers : {
                  "content-Type": "application/json"},
      body: JSON.Stringify ({
                  name:nam,
                  course:course
3)  3)
```

```
.then (response => response.json())
.then (data => {
                alert (data.message);

document.getElementById ("name").value = " ";
document.getElementById ("course").value = " ";
    displayStudents();
    });
}
function displayStudents () {
    fetch ("/students").then (response => response.json())
.then (students => {
    let list = document.getElementById ("studentlist");
    list.innerHTML = " ";
students.forEach (student => { let li =
    document.createElement ("li");
    li.innerHTML = student.id + " - " +
                student.name + " - " +
                student.course + "<button
onclick = 'deleteStudent (" + student.id + ")'>Delete
</button>";
    list.appendChild (li);
    });
});
}
function deleteStudent (id) {
    fetch ("/students/" + id, { method: "DELETE" })
.then (response => response.json())
```

```
.then (data => {
        alert (data.message);
        displayStudents ();
    });
}
displayStudents ();
</script>
</body>
</html>
```

O/P:-

Student Management System

Enter Student name

Enter Couse

Add student

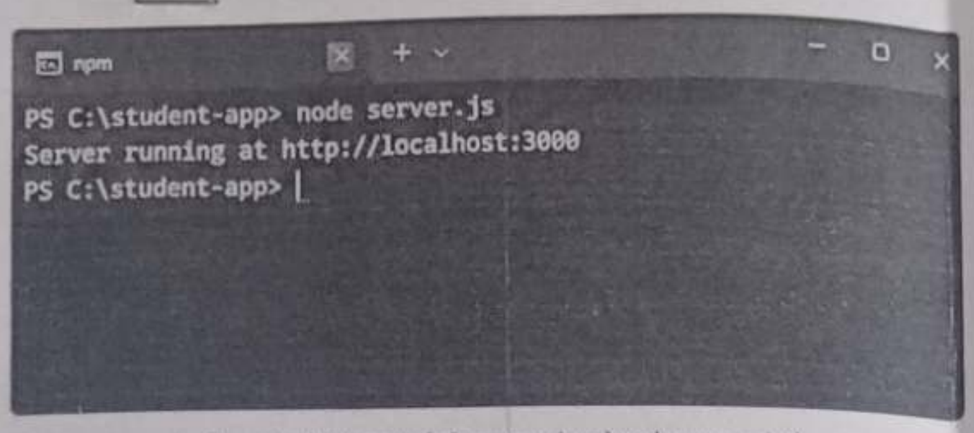
Student List:

1- Chaitrika - CSM delete

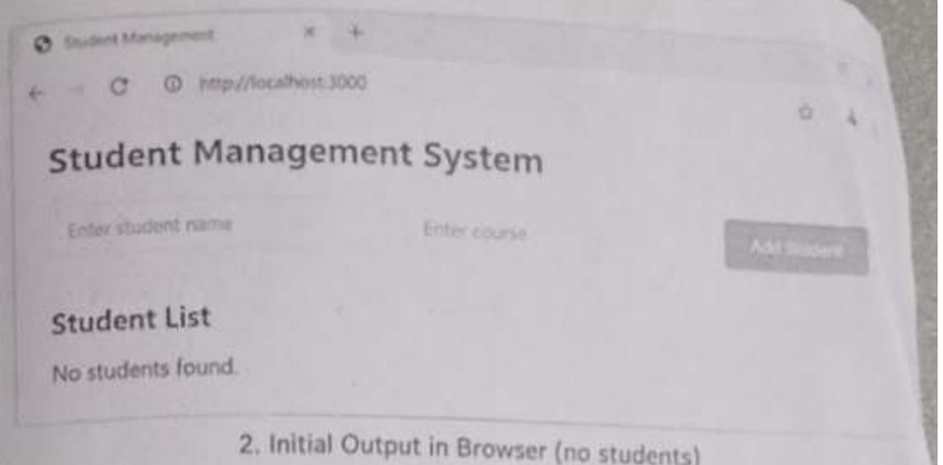
2- Kalyani - CSE delete

- The Express.js server is successfully started at http://localhost:3000

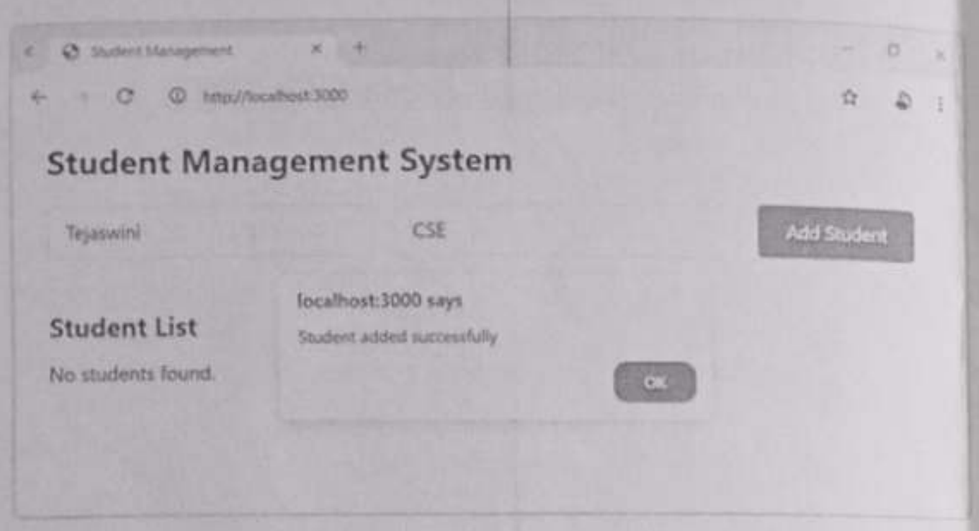
the brower displays the student management system with fields.



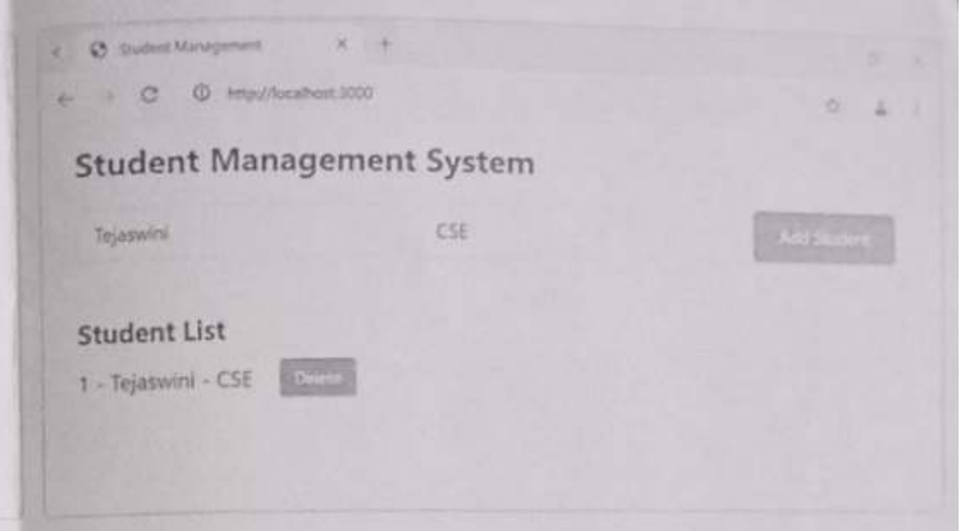
1. Terminal Output (after running `node server.js`)



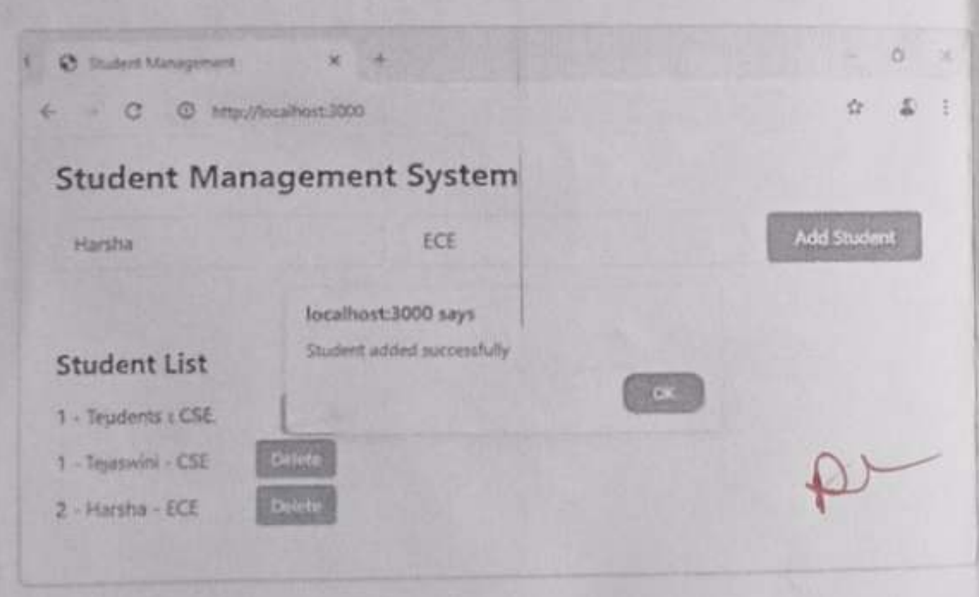
2. Initial Output in Browser (no students)



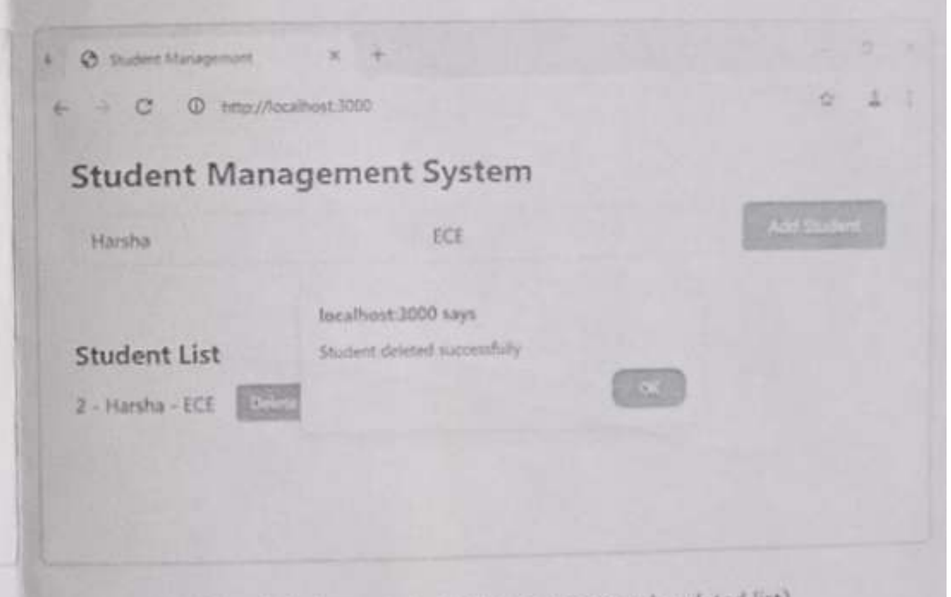
3. After adding a student (alert message)



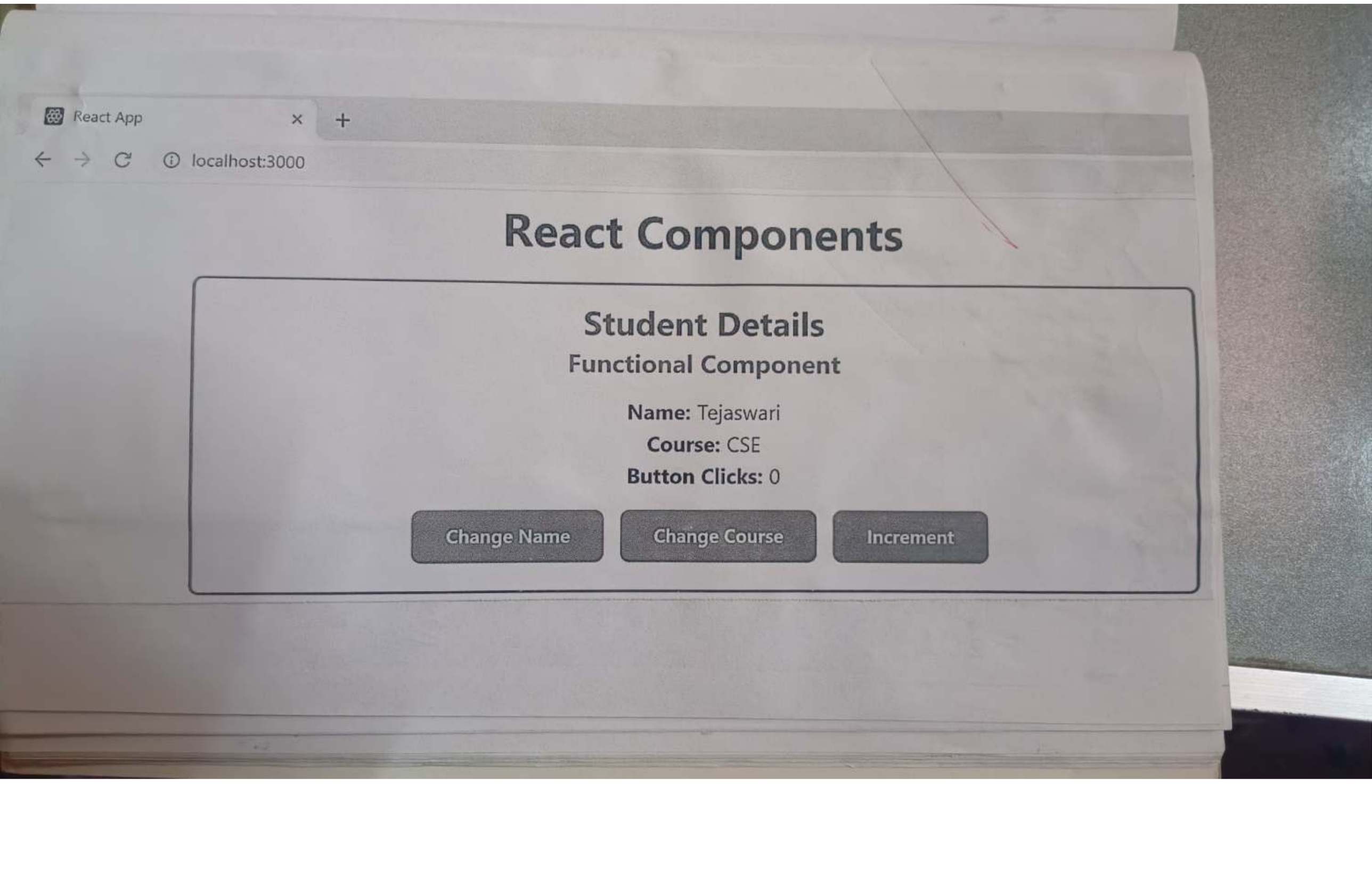
4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)


React App
localhost:3000
React Components
Student Details
Functional Component
Name: Tejaswari
Course: CSE
Button Clicks: 0
Change Name
Change Course
Increment

```
Const incrementCount = () => {
    setCount (count+1);
};
return (
    <div style={{
        border = " 8px solid blue",
        padding = "20px",
        margin = "20px",
        boardurRadius = "10px",
        textAlign = "center"
    }}>
    <h1> Student Details </h1>
    <h2> Functional Component </h2>
    <p> <b> Name: </b> {name} </p>
    <p> <b> Course : </b> {course} </p>
    <p> <b> Button clicks : </b> {count} </p>
    <button onclick = {changeName}>
        changeName </button>
    <button onclick = {changeCourse}> change
        Course </button>
    <button onclick = {incrementCount}> Increment
        Count </button>
    </div>
);
```

O/P:- When user clicks buttons, the name, course & count change dynamically without reloading the webpage

Experiment -7

1) Create a Functional Component & class component using react

Aim :- To develop a react application using functional components and class components to display student details, manage state, handle button clicks and dynamically update the webpage

Functional Component

It is a javascript function that returns JSX to display user interface. It uses React hooks such as usestate to manage state & update the webpage dynamically

Program:-

```
import React, {useState} from
'react';
function StudentFunctional(){
    const [name, setName] = useState("Chaitu");
    const [course, setCourse] = useState("CSM");
    const [count, setCount] = useState(0);
    const changeName = () => {
        setName("Pallavi");
    };
    const changeCourse = () => {
        setCourse("CSD")
    };
```

Class Component :-

A Class Component is an ES6 Javascript Class that extends React.Component. It uses the render() method to return JSX & manages state using this.state & this.setState()

Program :-

```
import React, {Component} from
'react';
class StudentClass extends Component{
    constructor(props){
        super(props);
        this.state={
            name = "Chaitrika",
            Course: "CSD",
            Count : 0
        };
    }
changeName = () => {
    this.setState({
        name: "Chaitu"
    });
};
changeCourse = () => {
    this.setState({
        course: "CSM"
    });
};
incrementCount = () => {
    this.setstate ((prevstate) => {
        Count : prevstate.count+1
```

```
    }));
  };
  render(){
    return(
      <div style={{
        border: "2px solid green",
        padding: "20px",
        margin: "20px",
        borderRadius: "10px",
        textAlign: "center"
      }}>
      <h1>Student Details</h1>
      <h2>Class Component</h2>
    <p><b>Name: </b>{this.state.name}</p>
    <p><b>Course: </b>{this.state.course}</p>
    <p><b>Button Clicks: </b>{this.state.count}</p>
  <button onClick={this.changeName}>
      ChangeName</button>
  <button onclick={this.changeCourse}>
      Change Course</button>
  <button onClick={this.incrementCount}>Increment
      </button>
    </div>
```

Student Details

Class Component

Name: Tejaswari

Course: CSE

Button Clicks: 0

Change Name | Change Course | Increment

The server start at :-

https://localhost:3000

The student records can be accessed using:

https://localhost:3000/students

olp:-

2. Implementation of HTTP methods:-

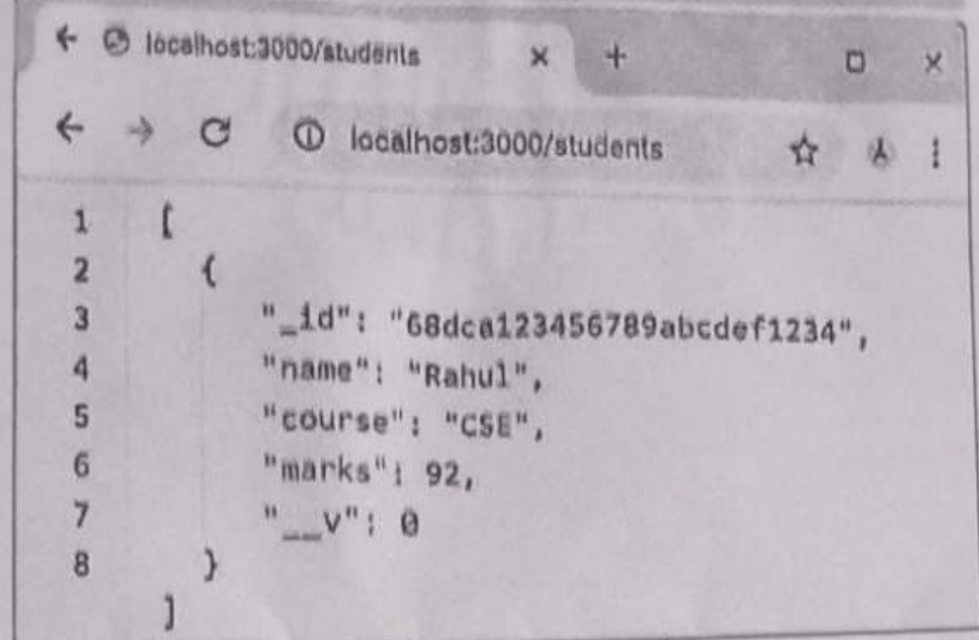
a. GET Method:- The GET method is used to retrive student records from JSON server

Code:-

```
fetch('https://localhost:3000/students')
  .then(response => response.json())
  .then(data => {
      console.log(data);
  })
  .catch(error => {
      console.log(error);
  });
```

olp:-

1. GET All Students (http://localhost:3000/students)



B. POST METHOD :- The post method is used to insert new student record into server.

Code :- fetch ("https://localhost:3000/students", {

method : 'POST',

headers : {

'content-type' : 'application/json'

},

body : JSON.stringify({

name : 'Rahul',

course : 'CSE',

marks : 92

})

})

.then (response => response.json())

.then (data => {

console.log (data);

})

.catch (error => {

console.log (error);

});

o/p :-

3. POST (Create Student) – Using Postman

localhost:3000/students

localhost:3000/students


```
{
    "name": "Tejaswari",
    "course": "AIML",
    "marks": 95,
    "_id": "68dcf9b7e4c9e8e1f3a6c5678",
    "__v": 0
}
```


c. PUT Method :- The PUT method is used to replace an existing student record with updated data.

Code :-

```
fetch (`https://localhost:3000/student/1`, {
    method: `PUT`,
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.Stringify({
        name: 'Tejaswari',
        Course: 'AIML',
        marks: 95
    })
})
.then(response => response.json())
.then(data => {
        console.log(data);
})
.catch(error => {
        console.log(error);
});
```

O/P :-

4. PUT (Update Student) – Using Postman

localhost:3000/students

localhost:3000/students/68dca123456789abcdef1234


```
{
    "_id": "68dca123456789abcdef1234",
    "name": "Rahul",
    "course": "AIML",
    "marks": 96,
    "__v": 0
}
```


D. PATCH Method :- It is used to update only selected fields of an existing student record.

Code :-

```
fetch('http://localhost:3000/students/2', {
    method : 'PATCH',
    headers : {
        'Content-type' : 'application/json'
    },
    body : JSON.stringify({
        marks : 96
    })
})
.then (response => response.json())
.then (data => {
    Console.log(data);
})
.catch (error => {
    Consol.log (error);
});
```

o/p:-

6. GET All Students (After Deletion)

localhost:3000/students

localhost:3000/students

```
1  []
2
```

E) DELETE Method:- It is used to remove an existing student record from the server.

Code:-
```
fetch ('http://localhost:3000/students/3', {
    method: 'DELETE'
})
.then (response => {
    if (response.ok) {
        console.log ("student deleted successfully");
    }
})
.catch (error => {
    Console.log (error);
});
```

o/p:-

5. DELETE (Remove Student) – Using Postman

localhost:3000/students

localhost:3000/students/68dca123456789abcdef1234

```
1  {
2      "message": "Student deleted successfully"
3  }
```

- After executing the POST, PUT, PATCH & DELETE Operations in sequence, the remaining student will be;

O/P:-

2. GET Student by ID (http://localhost:3000/students/68dca123456789abcdef1234)

localhost:3000/students

localhost:3000/students/68dca123456789abcdef1234


```
{
    "_id": "68dca123456789abcdef1234",
    "name": "Rahul",
    "course": "CSE",
    "marks": 92,
    "__v": 0
}
```


Thus, HTTP methods GET, POST, PUT, PATCH & DELETE were successfully implemented using JSON Server. The CRUD operations were performed on student records the data was retrived, inserted, updated & deleted using HTTP requests

(3)

Experiment - 9

9. Develop CRUD operations using MongoDB.

Aim:- To develop a web application using Node.js, Express & MongoDB to perform CRUD operations on Student record

Theory:- MongoDB is NOSQL database that stores data in flexible, JSON-like documents, CRUD operations are used to manage data in a database

Mongoose is an object Data Modeling (ODM) library for MongoDB & Node.js. It helps define Schemas, interact with collections & perform DB operations.

1. Installation & project setup:-

```
MKdir MongoDB_CRUD
Cd MongoDB_CRUD
npm init -y
npm install express mongoose
```

2. Program: Server.js

```
Const express = require('express');
Const mongoose = require('mongoose');
Const app = express();
Const PORT = 3000;
app.use(express.json());
Mongoose.connect('mongodb://127.0.0.1:27107/
                  College')
. then(()=> console.log("MongoDB connected"))
```

```
.catch (err => console.log(err));

//creati student
  const  studentSchema = new
  mongoose.schema({
        name: {
          type: String,
          required: true
        },
        course: {
          type: String,
          required: true
        },
        marks: {
          type: Number
          required: true
        }
    });

//creati student model
  const student = mongoose.model('Student', studentSchema);

//creati: Add
app.post('/students', async (req, res) => {
      try {
        const student = new student (req.body);
        const savedStudent = await student.save();
        res.status(201).json(savedStudent);
      } catch(error) {
        res.status(400).json({ message:
                              error: message });
      }
});
```

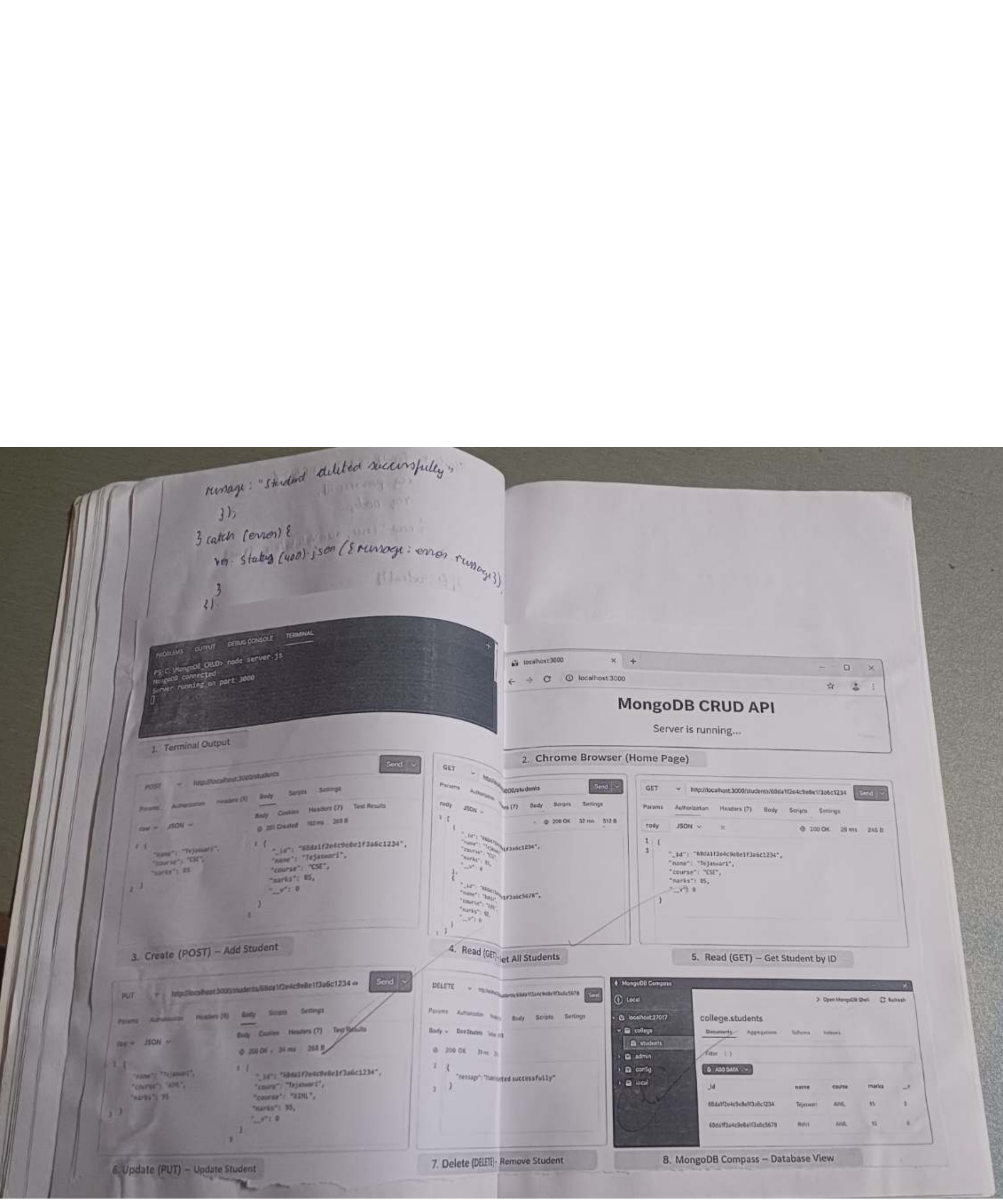
```
//READ: Get all students
app.get('/students', async(req, res) =>{
    try{
      Const students = await
      Student.find();
      res.status(500).json({message: error.message});
    }
});

//READ: Get Student by ID
app.get('/students/:id', async(req, res) =>{
    try{
        Const student = await
        student.findById(req.params.id);
        if(!student){
            return res.status(404).json({
            message:"Student not found"
            });
        }
    res.status(200).json(student);
    } catch(error){
      res.status(400).json({message:
        error.message});
    }
});

//UPDATE
app.put('/students/:id', async(req,res)=>{
    try{
      Const student = await
```

```
Student.findByIdAndUpdate(
    req.params.id,
    req.body,
  { new: true, runValidators: true }
);
if(!student){
    return res.status(404).json({
      message: "Student not found"
    });
}
res.status(200).json(student);
} catch(error){
    res.status(400).json({ message:
        error.message});
}});

//DELETE
app.delete('/students/:id', async (req, res) => {
  try{
    const student = await student.
        findByIdAndDelete(
            req.params.id
        );
    if(!student){
        return res.status(404).json({
          message: "student not found"
        });
    }
    res.status(200).json({
```


1. Terminal Output
2. Chrome Browser (Home Page)
3. Create (POST) – Add Student
4. Read (GET) – Get All Students
5. Read (GET) – Get Student by ID
6. Update (PUT) – Update Student
7. Delete (DELETE) – Remove Student
8. MongoDB Compass – Database View
MongoDB CRUD API
Server is running...